Adversarial Beta Domain Randomization for Robust Sim2Sim Transfer in Hopper Locomotion

Francesco Vanella
s349141
s349141@studenti.polito.it

Gabriele Imperiale
s346314
s346314@studenti.polito.it

Luciano Selimaj
s347565
s347565@studenti.polito.it

***Abstract*—In this paper, we investigate robustness and generalization in reinforcement learning for robotic locomotion under dynamics variations. We train and evaluate policies in a controlled sim-to-sim setting on the Gym Hopper task, comparing classic policy gradient and actor-critic methods with a modern off-policy baseline. We then analyze the impact of domain randomization strategies: we evaluate Uniform Domain Randomization (UDR) and propose an adaptive variant, Adversarial Beta Domain Randomization (AdvBeta), which reshapes the parameter sampling distribution toward failure-inducing regions via a percentile-based curriculum. Experiments show that AdvBeta achieves a $+16.8\%$ improvement in target-domain return over uniform randomization with reduced variance, highlighting the role of both algorithm choice and curriculum-driven dynamics randomization for robust generalization.**

1. Introduction

Reinforcement Learning (RL) is a machine learning paradigm in which an agent learns a policy by interacting with its environment, obtaining rewards as feedback to maximize the expected long-term return.

In robotics, RL is particularly useful because it allows the agent to learn complex behaviors without explicitly knowing the dynamical model of the system. This makes it suitable for high-dimensional, nonlinear control problems such as locomotion.

A central problem in this setting is the so-called *sim-to-real gap*: policies trained in simulation can degrade significantly when physical parameters in testing differ from those seen during training. Even small mismatches in physical parameters such as masses, friction coefficients, or actuator properties can lead to severe performance degradation.

In this work, we focus on simulated locomotion and study generalization in a controlled *sim-to-sim* setting, which serves as a proxy for sim-to-real transfer. Specifically, we train policies in the Gym *Hopper* environment and evaluate them under systematic source→target mismatch in physical parameters. This setup allows us to isolate and analyze the effect of dynamics variations while maintaining full experimental control and reproducibility. It also allows us to avoid all the problems related to the hardware of a real robot and the consequent costs.

Our contributions are twofold. First, we provide a comparative evaluation of representative RL algorithms, spanning from classic policy-gradient and actor-critic methods (REINFORCE and Actor-Critic) to a modern off-policy baseline, Soft Actor-Critic (SAC). These algorithms differ in terms of stability, sample efficiency, and bias-variance trade-offs, making them suitable candidates for analyzing robustness under dynamics shifts.

Second, we investigate robustness-oriented training techniques based on domain randomization. We start with Uniform Domain Randomization (UDR) and then, building upon this baseline, we introduce an extension, **Adversarial Beta Domain Randomization** (**AdvBeta**), which adaptively shapes the parameter distribution towards failure parameter configurations associated with low returns. This approach aims to further enhance the robustness and performance of policies in unseen dynamics. The source code is available in the GitHub repository.

2. Related Works

Deep Reinforcement Learning (DRL) has become a dominant paradigm for solving high-dimensional continuous control tasks in robotics. Early approaches relied on on-policy gradient methods, such as REINFORCE [1] and standard Actor-Critic architectures, which directly optimize the policy objective but often suffer from high variance updates and sample inefficiency. To address these limitations, trust region methods such as PPO [2] established a strong baseline for stability, although their on-policy nature restricts data reuse. In contrast, off-policy algorithms based on the maximum entropy framework, most notably Soft Actor-Critic (SAC) [3], have become strong solutions in a lot of scenarios, thanks to replay-buffer reuse and entropy-regularized objectives that encourage robust exploration.

Despite the theoretical strengths of these algorithms, deploying policies trained in simulation to physical systems remains challenging due to the *sim-to-real gap*. Domain Randomization (DR) has established itself as a powerful, zero-shot transfer technique to bridge this gap without requiring real-world data during training. The foundational approach, Uniform Domain Randomization (UDR) [4], [5], exposes the agent to a variety of simulated environments with randomized physical parameters drawn from fixed uniform distributions. The premise is that if the agent learns to adapt to a sufficiently wide distribution of dynamics, the target environment will appear as just another variation within the training distribution [6].

However, UDR often suffers from inefficiency, as uniform

sampling may dedicate significant training time to trivial or irrelevant parameter configurations. To overcome this, recent research has shifted towards *Active* or *Automatic* Domain Randomization strategies, which treat parameter selection as a curriculum learning problem. Adversarial approaches typically employ a distinct agent that actively perturbs the environment to minimize the primary agent's performance, driving the policy toward robust minimax solutions [7]. Other methods dynamically adjust the training distribution to maximize information gain or maintain the agent at the frontier of its capabilities. Our approach follows this line by using a simple percentile-based mechanism to bias the sampling distribution toward failure-inducing regions, aiming to increase robustness without introducing a fully separate adversary.

3. Methodology

This section describes the environment and the algorithms used in this work. We first introduce the Gym Hopper environment and its source–target dynamics mismatch, then present the policy gradient methods under evaluation: REINFORCE, Actor–Critic, and Soft Actor–Critic (SAC). The domain randomization strategies, UDR and AdvBeta, are described in the dedicated sections that follow.

3.1. Gym Hopper Environment

The Hopper is a monopedal simulated robot from the OpenAI Gym library, whose objective is to achieve forward locomotion while maintaining balance. At each time step, the reward signal is composed of three terms:

$$reward = forward_reward + alive_bonus - control_cost$$

- **Forward reward**: proportional to the velocity along the x-axis, it incentivizes forward progression.
- **Alive bonus**: a constant reward of 1.0 granted at each step in which the hopper remains upright. The episode terminates when a fall is detected (e.g., excessive torso inclination or insufficient height).
- **Control cost**: a quadratic penalty on the joint torques, $0.001 \cdot \|a\|_2^2$, where a is the 3-dimensional action vector. This term discourages high-magnitude motor commands, promoting energy-efficient and stable locomotion.

The agent's physical and operational characteristics are detailed below.

Physical structure: Hopper consists of four rigid bodies (torso, thigh, leg, and foot) connected by three actuated hinge joints.

State space: The state is an 11-dimensional continuous vector comprising a subset of generalized positions (e.g., torso height, torso angle, joint angles) and the corresponding velocities (joint/angular velocities and torso translational velocity).

Action space: The action is a 3-dimensional continuous vector with components in $[-1, 1]$, representing the torques applied at the three hinge joints.

Dynamics parameters: We study inertial variations by modifying link masses. In the Source environment, the masses of (torso, thigh, leg, foot) are approximately [2.57 4.06 2.78 5.32], while in the Target environment they are approximately [3.67 4.06 2.78 5.32], i.e., only the torso mass is shifted.

3.2. REINFORCE

REINFORCE is a Monte-Carlo policy-gradient method that updates the policy parameters θ from complete episodes [1]. Let $\pi_\theta(a \mid s)$ denote a stochastic policy. The objective is to maximize the expected discounted return

$$J(\theta) = \mathbb{E}_{\pi_\theta}\left[\sum_{t=0}^{T-1} \gamma^t r_t\right]. \tag{1}$$

Using the policy-gradient theorem, the gradient can be estimated from sampled trajectories as

$$\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta}\left[G_t \, \nabla_\theta \log \pi_\theta(a_t \mid s_t)\right], \tag{2}$$

where G_t is the (discounted) reward-to-go from time t:

$$G_t = \sum_{k=0}^{T-t-1} \gamma^k r_{t+k}. \tag{3}$$

From a single collected episode, REINFORCE performs the parameter update

$$\theta \leftarrow \theta + \alpha \, G_t \, \nabla_\theta \log \pi_\theta(a_t \mid s_t). \tag{4}$$

Baseline and variance reduction: Vanilla REINFORCE can have high variance. A standard variance-reduction technique is to subtract a baseline b (e.g., a constant or an estimate of the value function) without changing the expected gradient. Defining the advantage estimate $A_t = G_t - b$, the update becomes

$$\theta \leftarrow \theta + \alpha \, A_t \, \nabla_\theta \log \pi_\theta(a_t \mid s_t). \tag{5}$$

Equivalently, this corresponds to minimizing the weighted negative log-likelihood loss

$$\mathcal{L}(\theta) = -\sum_t A_t \, \log \pi_\theta(a_t \mid s_t). \tag{6}$$

3.3. Actor-Critic

The Actor-Critic (AC) method builds upon the policy gradient framework by jointly learning a policy and a value function. The value function assists the policy update by providing lower-variance advantage estimates, addressing a key limitation of vanilla policy gradient methods [8].

AC consists of two components [1]:

- **Critic** updates the value function parameters w. In our case the function is the state-value

$$V_w(s) \approx \mathbb{E}_\pi\left[\sum_{t=0}^{\infty} \gamma^t r(s_t, a_t) \;\middle|\; s_0 = s\right] \tag{7}$$

  which is the long-term value of being in state s following policy π.
- **Actor** updates the policy parameters θ for $\pi_\theta(a \mid s)$, in the direction suggested by the critic.

At each timestep, the critic estimates the state-value for the current state $V_w(s_t)$ and the next state $V_w(s_{t+1})$. The

advantage estimate, which quantifies whether the action taken was better or worse than expected, is computed as:

$$A_t = r_t + \gamma V_w(s_{t+1}) - V_w(s_t) \tag{8}$$

where r_t is the empirical reward. The training objective minimizes the actor loss

$$\mathcal{L}_{\text{actor}}(\theta) = -\ln \pi_\theta(a_t \mid s_t)\, A_t \tag{9}$$

and the critic loss

$$\mathcal{L}_{\text{critic}}(w) = A_t^2 = \big(r_t + \gamma V_w(s_{t+1}) - V_w(s_t)\big)^2. \tag{10}$$

The parameters are then updated via gradient descent:

$$\theta \leftarrow \theta - \alpha_a \nabla_\theta \mathcal{L}_{\text{actor}}(\theta) \tag{11}$$

$$w \leftarrow w - \alpha_c \nabla_w \mathcal{L}_{\text{critic}}(w). \tag{12}$$

Implementation details and hyperparameters (REINFORCE/Actor-Critic): Both methods use a Gaussian policy and a value baseline, with shared settings ($\gamma = 0.99$, Adam, learning rate 10^{-3}, MLP hidden size 64). REINFORCE is evaluated with baseline $b \in \{0, 20\}$. All hyperparameters are summarized in Table I.

TABLE I
REINFORCE/ACTOR-CRITIC HYPERPARAMETERS

Parameter	Value
γ	0.99
Optimizer / lr	Adam / 10^{-3}
Actor MLP	2×64 (`tanh`)
Critic MLP (AC)	2×64 (`ReLU`)
Policy std init / activation	0.5 / `softplus`
REINFORCE baseline b	0, 20

3.4. Soft Actor-Critic (SAC)

Soft Actor-Critic (SAC) is an off-policy actor-critic algorithm that bridges deep reinforcement learning with the *maximum entropy* framework [3]. Unlike standard RL, SAC maximizes a trade-off between the expected return and the policy entropy $\mathcal{H}$, encouraging exploration and robustness to estimation errors.

The optimal policy π^* maximizes the augmented objective:

$$J(\pi) = \sum_{t=0}^{T} \mathbb{E}_{(s_t,a_t)\sim\rho_\pi}\left[r(s_t,a_t) + \alpha\mathcal{H}(\pi(\cdot|s_t))\right]. \tag{13}$$

Soft Critic Update: SAC learns two Q-functions, $Q_{\theta_1}, Q_{\theta_2}$, to mitigate overestimation bias. To stabilize training, we first compute the soft Bellman target y:

$$y = r + \gamma(1-d)\left(\min_{j=1,2} Q_{\bar{\theta}_j}(s',a') - \alpha \log \pi_\phi(a'|s')\right), \tag{14}$$

where $a' \sim \pi_\phi(\cdot|s')$ and $\bar{\theta}$ denotes the target network weights. The parameters are then optimized by minimizing the squared error:

$$J_Q(\theta_i) = \mathbb{E}_{(s,a,r,s')\sim\mathcal{D}}\left[\left(Q_{\theta_i}(s,a) - y\right)^2\right]. \tag{15}$$

Soft Actor Update: The policy π_ϕ is updated by minimizing the Kullback-Leibler divergence. Let $\tilde{a}_\phi(s,\epsilon)$ denote the reparameterized action sampled via the squashed Gaussian. The objective is:

$$J_\pi(\phi) = \mathbb{E}_{s,\epsilon}\left[\alpha \log \pi_\phi(\tilde{a}_\phi|s) - \min_{j=1,2} Q_{\theta_j}(s,\tilde{a}_\phi)\right]. \tag{16}$$

This formulation allows the policy to act stochastically while remaining fully differentiable via the chain rule.

Rationale for choosing SAC over PPO: While Proximal Policy Optimization (PPO) [2] is widely regarded as a strong baseline for continuous control, its effectiveness is fundamentally limited by its *on-policy* nature. PPO mandates that gradient updates be computed solely using data collected by the current policy version, rendering previously collected trajectories unusable as soon as the policy parameters represent a new distribution. Consequently, this approach typically incurs high sample complexity, requiring extensive interaction with the environment to achieve convergence. In our experimental setting, where the interaction budget is strictly capped at 10^6 timesteps, *sample efficiency* becomes a critical constraint. Soft Actor-Critic (SAC) addresses this limitation through its off-policy formulation, which leverages a replay buffer to reuse past transitions for multiple gradient updates, thereby extracting significantly more learning signal from each interaction [3].

Moreover, in our setting with domain randomization and dynamics mismatch, the entropy regularized objective of SAC can discourage premature collapse to brittle deterministic behaviors, often leading to more robust locomotion policies.

Implementation details and hyperparameters: We use an MLP policy with two hidden layers of 256 units (ReLU) for both actor and critic networks. Training is run for 10^6 timesteps with replay buffer size 10^6, batch size 256, and learning starts after 10,000 steps. We use $\gamma = 0.99$, Polyak coefficient $\tau = 0.005$, and update targets every step. Entropy tuning is enabled (`ent_coef=auto`, `target_entropy=auto`). All hyperparameters are summarized in Table II.

TABLE II
SAC HYPERPARAMETERS

Parameter	Value
Total timesteps	10^6
Replay buffer size	10^6
Batch size	256
Learning starts	10,000
Train freq / gradient steps	1 / 1
Discount factor γ	0.99
Polyak coefficient τ	0.005
Target update interval	1
Entropy coefficient	auto
Target entropy	auto
Actor/Critic MLP	$[256, 256]$ (ReLU)

4. Uniform Domain Randomization (UDR)

Uniform Domain Randomization (UDR) improves robustness by sampling physical parameters from predefined distributions at the beginning of each episode, thereby training the policy on a family of dynamics rather than a single nominal model [4]–[6]. This is particularly useful in robotics, where imperfect system identification makes small inertial errors sufficient to destabilize a learned controller [9], [10].

In our Hopper benchmark, we randomize the masses of the thigh, leg, and foot links at every reset, while keeping the torso mass fixed. We do so to preserve a controlled source→target gap: the transfer test is defined by a systematic torso-mass shift, and randomizing the torso during training would partially overlap the target dynamics and make the evaluation less informative.

We sample each randomized mass uniformly within $\pm 30\%$ of its nominal value. This range is wide enough to expose the agent to meaningful inertial variability, yet bounded to avoid overly extreme dynamics that could make training unnecessarily conservative or unstable [7].

5. AdvBeta

In this section, we introduce Adversarial Beta Domain Randomization (AdvBeta), our learning framework designed with the aim to enhance the performance of reinforcement learning agents against domain parameter variations. AdvBeta is motivated by the limitations of UDR, which samples environmental parameters uniformly from a fixed interval. While simple and widely used, this strategy is inherently inefficient: as training progresses, large portions of the uniform distribution correspond to easy tasks that provide negligible gradient information, while hard cases, namely tasks that are not already solved, are sampled infrequently. AdvBeta addresses this limitation by actively shaping the sampling distribution. It operates on the hypothesis that *not all tasks are equally informative*. By continuously monitoring the agent's performance, the algorithm identifies critical regions of the physics parameters where the policy fails and dynamically adapts the sampling distributions to concentrate on these weaknesses.

5.1. Method

The core of AdvBeta is an adversarial mechanism that fits the sampling distribution to the agent's failure modes. We propose modeling each independent physical parameter ξ_i by mapping it to the normalized interval $[0, 1]$ and sampling from a Beta distribution, defined as $\tilde{\xi}_i \sim \text{Beta}(\alpha_i, \beta_i)$. The Beta distribution is particularly suited for physical parameter randomization for three key reasons:

- **Bounded support:** Existing methods often employ Gaussian distributions, which are unbounded and require truncation with respect to the physical constraints; instead, the Beta distribution is inherently bounded. This allows us to enforce constraints, without the bias introduced by truncation.
- **Shape flexibility:** By varying the shape parameters $\alpha, \beta \in [1, \infty)$, the distribution can smoothly transition between uniform ($\alpha = \beta = 1$), unimodal ($\alpha, \beta > 1$), and skewed shapes ($\alpha \neq \beta$). This representational capacity allows the algorithm to target failures that occur at the boundaries or, instead, in intermediate regions.
- **Analytical simplicity:** The parameters of the Beta distribution can be computed in closed form, allowing efficient updates in our scenario.

During training, we maintain a First-In-First-Out buffer $\mathcal{D} = \{(\tilde{\boldsymbol{\xi}}^{(k)}, R^{(k)})\}_{k=1}^{N}$ storing the parameters and the corresponding episodic rewards for the last N episodes.

At the beginning of training, we conduct a warmup phase of W episodes, sampling the dynamical parameters only from the uniform distributions, to ensure the agent acquires a baseline policy.

To define what constitutes a failure mode in a task-agnostic way, we employ a dynamic percentile threshold. Let J_p be the p-th percentile (e.g., 20%) of the rewards currently stored in $\mathcal{D}$. We extract the set of failure parameters $\mathcal{F}$ as:

$$\mathcal{F} = \left\{ \tilde{\boldsymbol{\xi}}^{(k)} \in \mathcal{D} \mid R^{(k)} \leq J_p \right\}. \tag{17}$$

This relative thresholding ensures that the definition of failure evolves as the agent improves, creating an automatic curriculum that constantly stores the remaining weaknesses.

Given the failure set, for each dimension i, we compute the sample mean μ_i and variance σ_i^2 of it. We then seek the Beta parameters $\hat{\alpha}_i, \hat{\beta}_i$ that best describe this failure distribution. Using the method of moments, we solve for the shape parameters:

$$\nu_i = \frac{\mu_i(1 - \mu_i)}{\sigma_i^2} - 1 \tag{18}$$

$$\hat{\alpha}_i = \mu_i \nu_i, \quad \hat{\beta}_i = (1 - \mu_i)\nu_i \tag{19}$$

This update aligns the sampling distribution with the observed failure modes. For example, if the agent consistently falls when the link mass is high, μ_i will be close to 1, resulting in $\hat{\alpha}_i > \hat{\beta}_i$, which skews the probability mass towards the upper bound. We note that our approach to computing distribution parameters is computationally lightweight.

Active modification of the training distribution can destabilize the learning process. To mitigate this, we introduce three regularization mechanisms:

- **Soft updates:** To prevent instability, we do not update the distribution parameters after every episode; instead, the update is performed every n episodes. This value is set to allow the failure buffer to be sufficiently refreshed, ensuring that the optimization relies on a consistent set of recent failures. Furthermore, at each step we employ a smooth update with a constant $\tau \in (0, 1]$:

$$\alpha_{t+1} \leftarrow (1-\tau)\alpha_t + \tau\hat{\alpha}_t, \quad \beta_{t+1} \leftarrow (1-\tau)\beta_t + \tau\hat{\beta}_t \tag{20}$$

  This ensures a smooth evolution of the parameter distributions.

- **Parameter constraints:** If the variance of failures σ_i^2 approaches zero, the estimated α, β would tend to infinity, causing the distribution to collapse into a single point. To prevent this, we clip the parameters $\alpha, \beta \in [1.0, \alpha_{max}]$. The lower bound of 1.0 prevents U-shaped distributions, ensuring that the distribution remains unimodal or uniform.
- **Mixture strategy:** By focusing exclusively on hard cases, the agent may unlearn the basic dynamics, causing catastrophic forgetting. To counter this, AdvBeta employs a mixture strategy. At the beginning of each episode, we select the sampling distribution according to a mixture ratio ρ:

$$\boldsymbol{\xi} \sim \begin{cases} \text{Beta}(\boldsymbol{\alpha}, \boldsymbol{\beta}) & \text{with probability } \rho \\ \mathcal{U}(0,1) & \text{with probability } 1-\rho \end{cases} \tag{21}$$

In our experiment, we train the control policy with the SAC algorithm.

The overall implementation is summarized in Algorithm 1.

Algorithm 1: Adversarial Beta Curriculum (AdvBeta)

Input: Buffer size N, warmup episodes W, update freq n, mixture ratio ρ, soft update τ, clipping α_{max}, percentile p

Output: Generalizable policy $\pi_{\tilde{\xi}}$

```
Initialize: α ← 1, β ← 1, D ← ∅, t ← 0
while Training do
    t ← t + 1
    if t < W or Uniform(0,1) > ρ then
        Sample ξ̃ ~ U(0,1)
    else
        Sample ξ̃ ~ Beta(α, β)
    end
    Map to physical space: ξ ← Normalize^{-1}(ξ̃)
    Execute episode with ξ, receive reward R
    Update buffer D ← D ∪ {(ξ̃, R)}
    if t ≥ W and t mod n = 0 then
        J_p ← Percentile(D, p)
        F ← {ξ̃^(k) | R^(k) ≤ J_p}
        foreach dimension i do
            Compute μ_i, σ_i^2 from F
            if σ_i^2 > 0 then
                Estimate (α̂_i, β̂_i) from (19)
                Clip to [1, α_max]
                Soft update (α_i, β_i) with (20)
            end
        end
    end
end
```

6. Experimental Results

All experiments in this section were run under a fixed computational budget. The on-policy methods (REINFORCE and Actor-Critic) and the SAC baselines (No-UDR and UDR) were evaluated over $N = 3$ random seeds due to computational resource constraints. The AdvBeta experiments were instead conducted over $N = 5$ random seeds in order to provide a more robust statistical analysis of the proposed method.

6.1. REINFORCE vs Actor-Critic

In this section, we compare the performance of REINFORCE and Actor-Critic (AC) algorithms. Training and testing were both conducted in the Source environment. Each algorithm was trained for 50,000 episodes and evaluated across three different random seeds to ensure reproducibility. For REINFORCE, we consider two variants: a vanilla version without baseline (RF_0) and a version with a fixed baseline of 20 (RF_20).

To analyze learning dynamics, we report the episode return during training, shown in Figure 1, Figure 2 and Figure 3. All methods exhibit a clear upward trend in the early stages of training, indicating an effective policy improvement from initially untrained policies.

As shown in Figure 1, RF_0 consistently achieves the lowest returns and displays high variability throughout training. The absence of a baseline results in high-variance gradient estimates, leading to unstable learning dynamics and frequent drops in episode return. Although RF_0 improves over time, it remains clearly dominated by the other approaches across the entire training horizon.

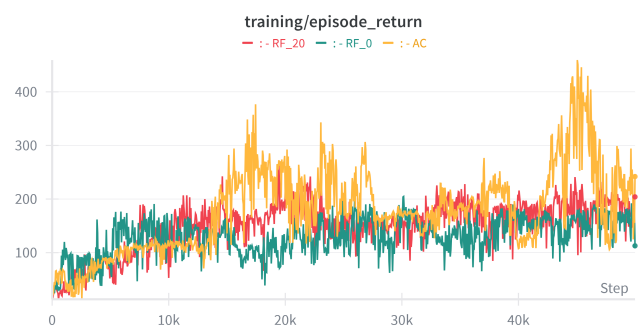

Fig. 1. Comparison between REINFORCE (RF_0), REINFORCE with baseline (RF_20), and Actor-Critic (AC)

Introducing a baseline in RF_20 significantly improves both performance and stability, as visible in Figure 2. Compared to RF_0, RF_20 achieves higher average returns and markedly smoother learning curves, confirming the expected variance-reduction effect of baselines in policy-gradient methods. The reduced oscillations indicate more consistent policy updates and improved convergence behavior.

The AC algorithm demonstrates competitive learning dynamics throughout training, as illustrated in Figure 1 and Figure 3. AC achieves the highest peak episode returns, occasionally surpassing RF_20. This behavior reflects the

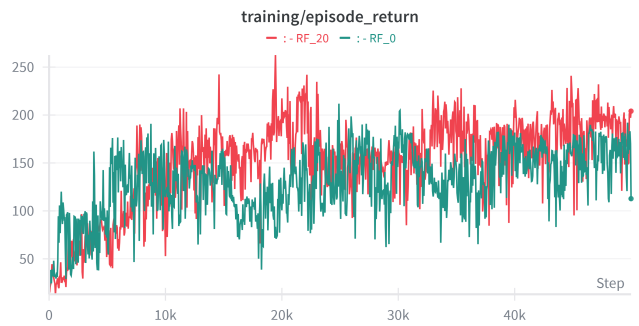

Fig. 2. Comparison between REINFORCE with and without baseline

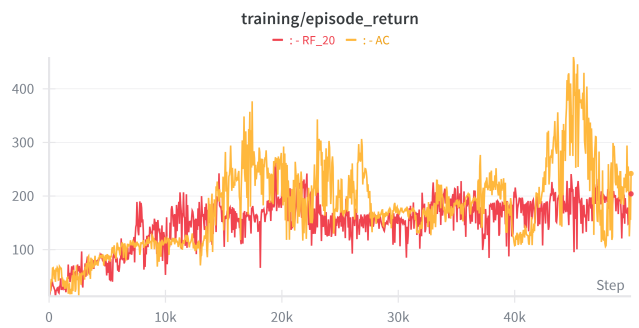

Fig. 3. Comparison between REINFORCE with baseline and Actor-Critic

advantage of jointly learning a value function, which provides lower-variance advantage estimates and enables faster policy refinement. However, these gains come at the cost of increased variability: AC exhibits larger oscillations during training, highlighting a trade-off between peak performance and stability.

TABLE III
REINFORCE AND ACTOR-CRITIC: MEAN REWARD AND INTRA-RUN STD OVER 50 TEST EPISODES ($N = 3$ SEEDS).

Algorithm	**Baseline**	**Mean $\pm$ Std$_{\text{intra}}$**
Actor-Critic	–	183.97 ± 36.95
REINFORCE	0	167.02 ± 22.69
REINFORCE	20	$\mathbf{201.31 \pm 11.46}$

Quantitative test results are reported in Table III which summarizes the mean episode return and the intra-run standard deviation computed over 50 test episodes, aggregated across three random seeds. These numerical results are consistent with the training dynamics observed in the figures.

Specifically, RF_20 achieves the highest mean return (201.31), while also exhibiting the lowest standard deviation (11.46), indicating that the learned policy is not only high-performing but also robust and consistent across episodes. This confirms, in numerical terms, the stabilizing effect of the baseline already observed in training curves.

The Actor-Critic method achieves a competitive mean return (183.97), outperforming RF_0 but remaining below RF_20. Notably, AC shows the largest standard deviation (36.95) among the three-methods, mirroring the higher variability observed during training. This suggests that, while AC can reach high returns, its performance is less consistent across episodes.

Finally, RF_0 yields the lowest mean return (167.02) and exhibits moderate variability (22.69). Despite a lower standard deviation than AC, its inferior average performance indicates limited convergence to a high-quality policy, confirming the detrimental impact of high-variance updates in the absence of a baseline.

6.2. Impact of Uniform Domain Randomization

To establish a rigorous benchmark for the Sim-to-Real transfer task, we evaluate the Soft Actor-Critic (SAC) algorithm under two training regimes: a deterministic training on nominal dynamics (No-UDR), and a training with Uniform Domain Randomization (UDR).

Quantification of the Reality Gap: The No-UDR baseline exhibits a pronounced overfitting to the nominal physics of the simulation. Although the agent achieves high returns in the Source environment ($R \approx 1700.51$), performance degrades catastrophically upon transfer to the Target domain. The discrepancy in physical parameters (specifically the unmodeled torso mass) results in a 35.06% drop in mean return, falling to 1104.32. This performance gap confirms that policies learned in static environments rely on specific inertial properties, lacking the robustness required for transfer.

Robustness via Randomization: The integration of UDR effectively mitigates the transfer loss. As detailed in Table IV, the UDR-trained agent achieves a mean target return of 1258.51, yielding a relative improvement of $+14.0\%$ over the non-randomized baseline. It is worth noting that randomization incurs a minor performance cost in the source domain (-1.4%). This phenomenon is consistent with the *robust control* literature: the policy sacrifices peak performance on nominal dynamics to maximize the worst-case performance across the distribution of possible dynamics.

The Stability-Reproducibility Trade-off: A granular analysis of the variance reveals an apparent trade-off introduced by UDR: it improves gait stability (intra-run) while exhibiting higher inter-seed variability. We note, however, that with only $N = 3$ random seeds the inter-seed standard deviation is a statistically limited estimator, and the observations reported below should be interpreted with appropriate caution.

- **Control Stability (Intra-run Analysis):** We define intra-run standard deviation of rewards as a metric for gait smoothness. The No-UDR agent shows high variance (97.08) in the target domain, indicative of oscillatory and unstable behavior. Conversely, UDR reduces this metric by a factor of $2.3\times$ (40.83). This suggests that exposure to diverse dynamics forces the policy to converge towards a conservative, low-variance locomotion strategy that is inherently more robust to perturbations.

- **Training Reproducibility (Inter-seed Analysis):** The inter-seed standard deviation rises from 344.98 (No-UDR) to 482.88 (UDR), a pattern consistent with a more complex optimization landscape under randomized dynamics. Among the three seeds, the best-performing run achieved strong generalization ($R > 1700$), while others failed to converge ($R \approx 769$). This polarization of outcomes suggests a potential sensitivity to initialization and sample collection; however, given the relative small number of seeds ($N = 3$), this finding cannot be considered statistically conclusive and may partly reflect the inherent stochasticity of training rather than a systematic effect of UDR.

TABLE IV
SAC NO-UDR VS. SAC UDR: MEAN REWARD AND STABILITY METRICS OVER 50 TEST EPISODES ($N = 3$ SEEDS).

Metric	Transfer Scenario	No-UDR	UDR	Δ Rel.
Performance (Mean Reward ↑)				
Mean Reward	Source → Source	1700.51	1675.98	−1.4%
Mean Reward	Source → Target	1104.32	**1258.51**	**+14.0%**
Stability Metrics (Std. Dev. ↓)				
Intra-run Std	Source → Target	97.08	**40.83**	−57.9%
Inter-seed Std	Source → Target	344.98	482.88	+40.0%

6.3. AdvBeta

Hyperparameter tuning of the failure percentile p*:* To balance performance with limited computational resources, the hyperparameters for the Adversarial Beta algorithm were mostly fixed based on preliminary empirical testing rather than extensive grid searches. Specifically, we set the buffer size to $N = 300$, the warmup phase to $W = 750$ episodes, and the update frequency to $n = 30$ episodes. W includes the SAC warmup phase and the buffer size to ensure that the agent has learned a baseline policy before starting our domain randomization strategy. The Beta parameters are clipped within $[1.0, 80.0]$ to prevent overfitting. We decided to heuristically set $\alpha_{max} = 80$ to enforce a minimum variance within the normalized parameter space $[0, 1]$, thus guaranteeing sufficient exploration around failure modes. A mix ratio of $\rho = 0.5$ is maintained throughout the process to ensure sufficient global exploration.

Instead, we performed a small tuning phase for the failure percentile p, as we identify it as one of the most critical hyperparameters in AdvBeta. We recall that this parameter defines the threshold for selecting hard parameters, because the algorithm updates the Beta distribution based on the top p-th percentile of environments where the agent obtained the lowest rewards.

We compared two cases:

- **Beta** ($p = 20$)**:** This case considers the worst 20% of episodes to be failures.
- **Beta** ($p = 10$)**:** This stricter case considers the worst 10% of episodes.

Table V presents the aggregated results over $N = 5$ random seeds. Although both configurations achieve comparable performance in the source environment, the stricter threshold ($p = 10$) yields superior generalization in the unseen target environment. By focusing on a narrower set of failures, the Beta distribution forces the agent to face the most critical physical parameters, resulting in a higher mean target reward (1302.51 vs 1246.47) and lower inter-seed variance.

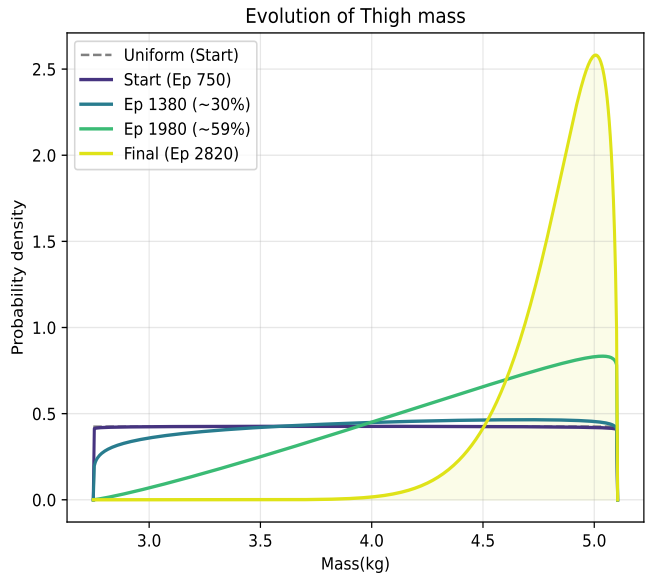

Fig. 4. Evolution of the Beta distribution for the thigh mass parameter in a representative run (seed 46). "Start", "30%", "60%", and "End" refer to the "adversarial" training phase.

TABLE V
ADVBETA FAILURE PERCENTILE TUNING: MEAN REWARD AND INTER-SEED STD OVER 50 TEST EPISODES (MEAN ± $\text{STD}_{\text{INTER}}$, $N = 5$ SEEDS).

Method	Source (Mean ± $\text{Std}_{\text{inter}}$)	Target (Mean ± $\text{Std}_{\text{inter}}$)
Beta ($p = 20$)	1661.29 ± 51.92	1246.47 ± 324.22
Beta ($p = 10$)	$\mathbf{1653.92 \pm 54.62}$	$\mathbf{1302.51 \pm 197.13}$

Evolution of Beta distributions*:* To show how the adversarial curriculum shapes the training distribution, we analyzed the evolution of the Beta densities for the physical parameters. Figure 4 illustrates the evolution of the sampling distribution for the thigh parameter, for a representative run (seed 46) using the best configuration (Beta $p = 10$). In particular, it represents the distributions at the start, 30%, 60%, at the end of the "adversarial" training. The plot shows how the distributions identify the hard tasks for the current policy.

Comparison with UDR*:* Finally, we compared our best model (Beta $p = 10$) with the standard UDR. As shown in Table VI, our method significantly outperforms the uniform baseline. The Beta agent with $p = 10$ achieves a Target Reward of 1302.51, an improvement of approximately +16.8% over UDR (1114.71). Furthermore, the reduced inter-seed standard deviation indicates that the policy learned with AdvBeta

domain randomization leads to greater training reproducibility and robustness.

TABLE VI
ADVBETA VS. UDR: MEAN REWARD AND INTER-SEED STD OVER 50 TEST EPISODES (MEAN $\pm$ $\text{STD}_{\text{INTER}}$, $N = 5$ SEEDS).

Method	Source (Mean $\pm$ $\text{Std}_{\text{inter}}$)	Target (Mean $\pm$ $\text{Std}_{\text{inter}}$)
UDR	1647.87 ± 55.01	1114.71 ± 394.23
Beta ($p = 10$)	$\mathbf{1653.92 \pm 54.62}$	$\mathbf{1302.51 \pm 197.13}$

7. Conclusion

In this work, we investigated robustness and generalization in reinforcement learning for robotic locomotion under dynamics variations, using a controlled sim-to-sim transfer benchmark based on the Gym Hopper environment. By systematically introducing mismatches in physical parameters between training and evaluation, we were able to analyze how different algorithmic choices and training strategies affect policy performance and stability under unseen dynamics.

Our comparative analysis of classic policy-gradient and actor-critic methods highlighted the importance of variance reduction mechanisms. In particular, REINFORCE with a fixed baseline achieved the best trade-off between average performance and stability among on-policy methods, confirming that reducing gradient variance plays a key role in learning consistent locomotion behaviors. Actor-Critic methods, while capable of reaching high returns, exhibited larger intra-run variability, reflecting a trade-off between peak performance and robustness.

Moving to modern off-policy approaches, our results clearly showed that policies trained without domain randomization suffer from severe performance degradation when transferred to the target environment, confirming the presence of a significant dynamics-induced reality gap even in simulation. Uniform domain randomization effectively mitigated this gap, improving target-domain performance and substantially reducing intra-run variability, at the cost of a small reduction in nominal performance and increased sensitivity across random seeds.

Building upon this baseline, we introduced Adversarial Beta Domain Randomization, which adaptively focuses training on failure-inducing regions of the parameter space. Experimental results demonstrated that this targeted randomization strategy consistently outperforms uniform domain randomization, yielding higher average returns and lower variance in the target environment. In particular, a stricter failure selection criterion led to the most robust policies, suggesting that concentrating learning on the most challenging dynamics is an effective way to improve generalization.

In general, these findings emphasize that robustness in robotic reinforcement learning emerges from the combination of sample-efficient algorithms and carefully designed training distributions. Although our study is limited to a sim-to-sim setting and to a constrained computational budget, the proposed methodology and insights provide a useful starting point for future work.

References

[1] L. Weng, "Policy gradient algorithms," *lilianweng.github.io*, 2018. [Online]. Available: https://lilianweng.github.io/posts/2018-04-08-policy-gradient/

[2] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *arXiv preprint arXiv:1707.06347*, 2017.

[3] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, "Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor," *arXiv preprint arXiv:1801.01290*, 2018.

[4] J. Tobin, R. Fong, A. Ray, J. Schneider, W. Zaremba, and P. Abbeel, "Domain randomization for transferring deep neural networks from simulation to the real world," *arXiv preprint arXiv:1703.06907*, 2017.

[5] X. B. Peng, M. Andrychowicz, W. Zaremba, and P. Abbeel, "Sim-to-real transfer of robotic control with dynamics randomization," in *2018 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2018, pp. 3803–3810.

[6] S. H"ofer, K. Bekris, A. Handa, J. C. Gamboa, F. Golemo, M. Mozifian, M. White *et al.*, "Perspectives on sim2real transfer for robotics: A summary of the r:ss 2020 workshop," *arXiv preprint arXiv:2012.09868*, 2020.

[7] F. Muratore, T. Gruner, M. Wiese, B. Belousov, M. Gienger, J. Peters, and L. Righetti, "Robot learning from randomized simulations: A review," *Frontiers in Robotics and AI*, vol. 9, p. 1051117, 2022.

[8] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. Cambridge, MA, USA: The MIT Press, 2018.

[9] J. Kober, J. A. Bagnell, and J. Peters, "Reinforcement learning in robotics: A survey," *The International Journal of Robotics Research*, vol. 32, no. 11, pp. 1238–1274, 2013.

[10] P. Kormushev, S. Calinon, and D. G. Caldwell, "Reinforcement learning in robotics: Applications and real-world challenges," in *Robotics Research*, ser. Springer Tracts in Advanced Robotics, vol. 114. Springer, 2016, pp. 531–547.